📘 Guía Maestra Definitiva: CRUD Full-Stack (.NET 10 Web API + Angular)

> **Manual de Estudio y Plantilla Genérica para Exámenes (Patrón Maestro-Detalle con Barra Lateral)**
> *Diseñado para adaptar cualquier entidad en menos de 10 minutos cambiando nombres de variables.*

📑 Tabla de Contenidos

1. Mapa Mental y Arquitectura del Flujo
2. El Patrón de Diseño con Franja Lateral (Sidebar)
3. Parte 1: Backend (.NET Core Web API)
4. 3.1 Modelos C# (`Cabecera.cs` y `Detalle.cs`)
5. 3.2 Capa de Acceso a Datos (`DADetalle.cs` y `DACabecera.cs`)
6. 3.3 Controladores API REST (`BackEjecucionesController.cs` y `CabecerasController.cs`)
7. 3.4 Configuración de `Program.cs` (CORS y API)
8. Parte 2: Frontend (Angular Standalone con Barra Lateral)
9. 4.1 Configurar `HttpClient` en `app.config.ts`
10. 4.2 Estilos Globales en `index.html`
11. 4.3 Modelos en TypeScript (`cabecera.model.ts` y `detalle.model.ts`)
12. 4.4 Servicio HTTP (`ejecuciones.service.ts`)
13. 4.5 Lógica del Componente con Sidebar y "Ver Más" (`panel.ts`)
14. 4.6 Vista con Barra Lateral, Tablas y Modales (`panel.html`)
15. 4.7 Estilos del Sidebar (`panel.css`)
16. Tabla Rápida de Reemplazo para el Examen
17. Top 5 Errores Típicos de Examen y Soluciones en 10 Segundos

---

1. Mapa Mental y Arquitectura del Flujo

```
[ Navegador Web ]
       │
       ▼ (1) Usuario selecciona "Cabecera" o "Detalle" en la Franja Izquierda
[ Componente Panel (.html + .ts + .css) ]
       │  • Barra lateral (Sidebar): cambia la variable seccionActiva
       │  • Tabla dinámica: muestra Cabeceras o Detalles según el clic
       │  • Acciones por fila: [Ver Más / Detallar], [Editar], [Eliminar]
       ▼ (2) Llama a métodos del Servicio
[ Servicio Angular (.service.ts) ]
       │  • Usa inject(HttpClient) para emitir peticiones asíncronas
       ▼ (3) Petición HTTP (GET, POST, PUT, DELETE) con JSON
[ Internet / Red Local ] ──► Autorizado por CORS en Program.cs
       │
       ▼ (4) Endpoints /api/ejecuciones y /api/cabeceras
[ Controladores .NET ([ApiController]) ]
       │
       ▼ (5) Métodos de Datos (DA / Entity Framework Core)
[ Base de Datos SQL Server ]
```

---

2. El Patrón de Diseño con Franja Lateral (Sidebar)

En evaluaciones de desarrollo de software empresarial, los profesores suelen solicitar una **interfaz de tipo Dashboard / Panel de Control**:

1. **Franja Lateral Izquierda (Sidebar)**:
- Permite alternar entre la entidad principal (**Cabecera / Clientes / Facturas**) y la entidad hija (**Detalles / Líneas / Ejecuciones**).
- Muestra insignias (*badges*) con la cantidad de registros en tiempo real.
2. **Área Principal (Derecha)**:
- Cabecera con título dinámico y botón de creación.
- Tarjetas de resumen métrico (KPIs: totales, recaudación, etc.).
- Filtro de búsqueda y paginador configurable (5, 10, 20 por página).
- Tabla interactiva con botones:
- 👁️ **Ver Más / Detallar**: Abre un modal informativo con todos los campos desglosados (incluyendo relaciones foráneas).
- 🖍️ **Editar**: Abre el formulario cargando los datos del registro.
- 🗑️ **Eliminar**: Pide confirmación y borra vía HTTP DELETE.

---

3. Parte 1: Backend (.NET Core Web API)

3.1 Modelos C# (`Cabecera.cs` y `Detalle.cs`)

Ubicación habitual: `Models/CabeceraOperacion.cs` y `Models/DetalleOperacion.cs`

```cs
// Models/CabeceraOperacion.cs
using System.ComponentModel.DataAnnotations;
using System.Text.Json.Serialization;

namespace BackendWebApp.Models
{
    public class CabeceraOperacion
    {
        [Key]
        public int IdCabeceraOperacion { get; set; }

        [Required]
        [MaxLength(100)]
        public string Cliente { get; set; } = string.Empty;

        public DateTime Registro { get; set; }

        [JsonIgnore] // Evita ciclos al serializar a JSON
        public virtual ICollection<DetalleOperacion>? DetalleOperacions { get; set; }
    }
}
```

```cs
// Models/DetalleOperacion.cs
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace BackendWebApp.Models
{
    public class DetalleOperacion
    {
        [Key]
        public int IdDetalleOperacion { get; set; }

        [Required]
        [MaxLength(100)]
        public string Descripcion { get; set; } = string.Empty;

        [Required]
        public int Cantidad { get; set; }

        [Required]
        public float Importe { get; set; }

        public float Igv { get; set; }

        public float Total { get; set; }

        // Clave Foránea
        public int IdCabeceraOperacion { get; set; }

        [ForeignKey("IdCabeceraOperacion")]
        public virtual CabeceraOperacion? CabeceraOperacion { get; set; }
    }
}
```

3.2 Capa de Acceso a Datos (`DADetalle.cs` y `DACabecera.cs`)

```
// Data/AccesoDatos/DADetalleEjecuciones.cs
using BackendWebApp.Models;
using Microsoft.EntityFrameworkCore;

namespace BackendWebApp.Data.AccesoDatos
{
    public class DADetalleEjecuciones
    {
        public IEnumerable<DetalleOperacion> GetDetalleOperacion()
        {
            using (var db = new ApplicationDbContext())
            {
                return db.DetalleOperacion.Include(x => x.CabeceraOperacion).ToList();
            }
        }

        public DetalleOperacion? GetDetalleOperacionById(int id)
        {
            using (var db = new ApplicationDbContext())
            {
                return db.DetalleOperacion.Include(x => x.CabeceraOperacion)
                                          .FirstOrDefault(x => x.IdDetalleOperacion == id);
            }
        }

        public int InsertDetalleOperacion(DetalleOperacion entidad)
        {
            using (var db = new ApplicationDbContext())
            {
                entidad.CabeceraOperacion = null; // Evita conflicto de entidad foránea
                db.Add(entidad);
                db.SaveChanges();
                return entidad.IdDetalleOperacion;
            }
        }

        public bool UpdateDetalleOperacion(DetalleOperacion entidad)
        {
            using (var db = new ApplicationDbContext())
            {
                var existente = db.DetalleOperacion.Find(entidad.IdDetalleOperacion);
                if (existente == null) return false;

                existente.Descripcion = entidad.Descripcion;
                existente.Cantidad = entidad.Cantidad;
                existente.Importe = entidad.Importe;
                existente.Igv = entidad.Igv;
                existente.Total = entidad.Total;
                existente.IdCabeceraOperacion = entidad.IdCabeceraOperacion;

                db.SaveChanges();
                return true;
            }
        }

        public bool DeleteDetalleOperacion(int id)
        {
            using (var db = new ApplicationDbContext())
            {
                var registro = db.DetalleOperacion.Find(id);
                if (registro == null) return false;

                db.DetalleOperacion.Remove(registro);
                db.SaveChanges();
```

```
                return true;
            }
        }
    }
}
```

```
// Data/AccesoDatos/DACabeceraEjecuciones.cs
using BackendWebApp.Models;

namespace BackendWebApp.Data.AccesoDatos
{
    public class DACabeceraEjecuciones
    {
        public IEnumerable<CabeceraOperacion> GetCabeceraOperacion()
        {
            using (var db = new ApplicationDbContext())
            {
                return db.CabeceraOperacion.ToList();
            }
        }
    }
}
```

3.3 Controladores API REST (`BackEjecucionesController.cs` y `CabecerasController.cs`)

```
// Controllers/BackEjecucionesController.cs
using BackendWebApp.Data.AccesoDatos;
using BackendWebApp.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Cors;
using Microsoft.AspNetCore.Mvc;

namespace BackendWebApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Route("api/ejecuciones")]
    [AllowAnonymous]
    [EnableCors("AngularPolicy")]
    public class BackEjecucionesController : ControllerBase
    {
        private readonly DADetalleEjecuciones _da = new DADetalleEjecuciones();

        [HttpGet]
        public IActionResult Listar() => Ok(_da.GetDetalleOperacion());

        [HttpGet("{id}")]
        public IActionResult ObtenerPorId(int id)
        {
            var item = _da.GetDetalleOperacionById(id);
            return item == null ? NotFound() : Ok(item);
        }

        [HttpPost]
        public IActionResult Crear([FromBody] DetalleOperacion entidad)
        {
            if (!ModelState.IsValid) return BadRequest(ModelState);

            entidad.IdDetalleOperacion = 0;
            entidad.Igv = entidad.Importe * 0.18f;      // Cálculo del IGV
            entidad.Total = entidad.Importe + entidad.Igv; // Total = Importe + IGV

            var id = _da.InsertDetalleOperacion(entidad);
            if (id <= 0) return BadRequest("Error al insertar");

            entidad.IdDetalleOperacion = id;
            return CreatedAtAction(nameof(ObtenerPorId), new { id }, entidad);
        }

        [HttpPut("{id}")]
        public IActionResult Actualizar(int id, [FromBody] DetalleOperacion entidad)
        {
            if (id != entidad.IdDetalleOperacion) return BadRequest();
            if (!ModelState.IsValid) return BadRequest(ModelState);

            entidad.Igv = entidad.Importe * 0.18f;
            entidad.Total = entidad.Importe + entidad.Igv;

            var ok = _da.UpdateDetalleOperacion(entidad);
            return ok ? Ok(entidad) : NotFound();
        }

        [HttpDelete("{id}")]
        public IActionResult Eliminar(int id)
        {
            var ok = _da.DeleteDetalleOperacion(id);
            return ok ? Ok() : NotFound();
        }
```

```
    }
}
```

```
// Controllers/CabecerasController.cs
using BackendWebApp.Data.AccesoDatos;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Cors;
using Microsoft.AspNetCore.Mvc;

namespace BackendWebApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Route("api/cabeceras")]
    [AllowAnonymous]
    [EnableCors("AngularPolicy")]
    public class CabecerasController : ControllerBase
    {
        [HttpGet]
        public IActionResult Listar()
        {
            var da = new DACabeceraEjecuciones();
            return Ok(da.GetCabeceraOperacion());
        }
    }
}
```

3.4 Configuración de `Program.cs` (CORS y API)

```
using BackendWebApp.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Conexión SQL Server
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

// Controladores con manejo de ciclos JSON
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.ReferenceHandler = System.Text.Json.Serialization.ReferenceHa
        options.JsonSerializerOptions.DefaultIgnoreCondition = System.Text.Json.Serialization.JsonI
    });

// CORS: Permitir comunicación con localhost:4200 (Angular)
builder.Services.AddCors(options =>
{
    options.AddPolicy("AngularPolicy", policy =>
    {
        policy.WithOrigins("http://localhost:4200")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

var app = builder.Build();

app.UseRouting();
app.UseCors("AngularPolicy"); // Obligatorio antes de MapControllers
app.UseAuthorization();
app.MapControllers();

app.Run();
```

---

4. Parte 2: Frontend (Angular Standalone con Barra Lateral)

4.1 Configurar `HttpClient` en `app.config.ts`

Ubicación: `src/app/app.config.ts`

```typescript
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient() // <-- Imprescindible para llamadas API
  ]
};
```

4.2 Estilos Globales en `index.html`

Ubicación: `src/index.html`

```html
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Sistema de Operaciones</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">

  <!-- Bootstrap 5 CSS y Bootstrap Icons -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesh
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-i
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

4.3 Modelos en TypeScript (`cabecera.model.ts` y `detalle.model.ts`)

```typescript
// src/app/models/cabecera.model.ts
export interface CabeceraOperacion {
  idCabeceraOperacion: number;
  cliente: string;
  registro?: string;
}
```

```typescript
// src/app/models/detalle.model.ts
import { CabeceraOperacion } from './cabecera.model';

export interface DetalleOperacion {
  idDetalleOperacion?: number; // '?' para permitir creación sin ID
  descripcion: string;
  cantidad: number;
  importe: number;
  igv?: number;                // Calculado en backend
  total?: number;              // Calculado en backend
  idCabeceraOperacion: number;
  cabeceraOperacion?: CabeceraOperacion;
}
```

4.4 Servicio HTTP (`ejecuciones.service.ts`)

Ubicación: `src/app/services/ejecuciones.services.ts`

```typescript
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { DetalleOperacion } from '../models/detalle.model';
import { CabeceraOperacion } from '../models/cabecera.model';

@Injectable({
  providedIn: 'root'
})
export class EjecucionesService {
  private http = inject(HttpClient);

  private apiUrl = 'http://localhost:5264/api/ejecuciones';
  private cabecerasUrl = 'http://localhost:5264/api/cabeceras';

  // GET Operaciones
  getEjecuciones(): Observable<DetalleOperacion[]> {
    return this.http.get<DetalleOperacion[]>(this.apiUrl);
  }

  // GET Operación por ID
  getEjecucionById(id: number): Observable<DetalleOperacion> {
    return this.http.get<DetalleOperacion>(`${this.apiUrl}/${id}`);
  }

  // GET Cabeceras (Clientes)
  getCabeceras(): Observable<CabeceraOperacion[]> {
    return this.http.get<CabeceraOperacion[]>(this.cabecerasUrl);
  }

  // POST Crear
  crearEjecucion(op: DetalleOperacion): Observable<DetalleOperacion> {
    return this.http.post<DetalleOperacion>(this.apiUrl, op);
  }

  // PUT Actualizar
  actualizarEjecucion(id: number, op: DetalleOperacion): Observable<any> {
    return this.http.put<any>(`${this.apiUrl}/${id}`, op);
  }

  // DELETE Eliminar
  eliminarEjecucion(id: number): Observable<any> {
    return this.http.delete<any>(`${this.apiUrl}/${id}`);
  }
}
```

4.5 Lógica del Componente con Sidebar y "Ver Más" (`panel.ts`)

Ubicación: `src/app/components/panel/panel.ts`

```ts
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { EjecucionesService } from '../../services/ejecuciones.services';
import { DetalleOperacion } from '../../models/detalle.model';
import { CabeceraOperacion } from '../../models/cabecera.model';

@Component({
  selector: 'app-panel',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './panel.html',
  styleUrl: './panel.css'
})
export class PanelComponent implements OnInit {
  private ejecucionesService = inject(EjecucionesService);

  // Variable clave para alternar secciones desde el Sidebar
  seccionActiva: 'detalles' | 'cabeceras' = 'detalles';

  // Colecciones de datos
  ejecuciones: DetalleOperacion[] = [];
  cabeceras: CabeceraOperacion[] = [];
  cargando: boolean = true;
  filtroTexto: string = '';

  // Paginación
  paginaActual: number = 1;
  registrosPorPagina: number = 5;

  // Modales
  mostrarModal: boolean = false;
  modoEdicion: boolean = false;
  operacionEdicion: DetalleOperacion = this.limpiarFormulario();

  // Modal "Ver Más / Detallar"
  mostrarModalVerMas: boolean = false;
  detalleSeleccionado: DetalleOperacion | null = null;
  cabeceraSeleccionada: CabeceraOperacion | null = null;

  ngOnInit(): void {
    this.cargarDatos();
    this.cargarCabeceras();
  }

  cargarDatos(): void {
    this.cargando = true;
    this.ejecucionesService.getEjecuciones().subscribe({
      next: (datos) => {
        this.ejecuciones = datos;
        this.cargando = false;
      },
      error: (err) => {
        console.error(err);
        this.cargando = false;
      }
    });
  }

  cargarCabeceras(): void {
    this.ejecucionesService.getCabeceras().subscribe({
      next: (datos) => (this.cabeceras = datos),
      error: (err) => console.error(err)
    });
  }
```

```
  // Alternar entre Cabeceras y Detalles
  cambiarSeccion(seccion: 'detalles' | 'cabeceras'): void {
    this.seccionActiva = seccion;
    this.paginaActual = 1;
    this.filtroTexto = '';
  }

  // Paginación y Filtrado para Detalles
  get detallesFiltrados(): DetalleOperacion[] {
    if (!this.filtroTexto) return this.ejecuciones;
    const txt = this.filtroTexto.toLowerCase();
    return this.ejecuciones.filter(
      item => item.descripcion.toLowerCase().includes(txt) ||
              item.cabeceraOperacion?.cliente?.toLowerCase().includes(txt)
    );
  }

  get detallesPaginados(): DetalleOperacion[] {
    const inicio = (this.paginaActual - 1) * this.registrosPorPagina;
    return this.detallesFiltrados.slice(inicio, inicio + this.registrosPorPagina);
  }

  // Paginación y Filtrado para Cabeceras
  get cabecerasFiltradas(): CabeceraOperacion[] {
    if (!this.filtroTexto) return this.cabeceras;
    const txt = this.filtroTexto.toLowerCase();
    return this.cabeceras.filter(c => c.cliente?.toLowerCase().includes(txt));
  }

  get cabecerasPaginadas(): CabeceraOperacion[] {
    const inicio = (this.paginaActual - 1) * this.registrosPorPagina;
    return this.cabecerasFiltradas.slice(inicio, inicio + this.registrosPorPagina);
  }

  get totalPaginas(): number {
    const total = this.seccionActiva === 'detalles' ? this.detallesFiltrados.length : this.cabecera
    return Math.ceil(total / this.registrosPorPagina) || 1;
  }

  cambiarPagina(p: number): void {
    if (p >= 1 && p <= this.totalPaginas) this.paginaActual = p;
  }

  // Métricas
  get totalMonto(): number {
    return this.ejecuciones.reduce((acum, item) => acum + (item.total || 0), 0);
  }
  get totalIgv(): number {
    return this.ejecuciones.reduce((acum, item) => acum + (item.igv || 0), 0);
  }

  // Acción "Ver Más / Detallar"
  abrirVerMasDetalle(item: DetalleOperacion): void {
    this.detalleSeleccionado = item;
    this.cabeceraSeleccionada = null;
    this.mostrarModalVerMas = true;
  }

  abrirVerMasCabecera(cab: CabeceraOperacion): void {
    this.cabeceraSeleccionada = cab;
    this.detalleSeleccionado = null;
    this.mostrarModalVerMas = true;
  }

  cerrarModalVerMas(): void {
    this.mostrarModalVerMas = false;
  }
```

```
  obtenerOperacionesPorCabecera(idCabecera: number): DetalleOperacion[] {
    return this.ejecuciones.filter(x => x.idCabeceraOperacion === idCabecera);
  }

  // CRUD
  abrirModalCrear(): void {
    this.modoEdicion = false;
    this.operacionEdicion = this.limpiarFormulario();
    this.mostrarModal = true;
  }

  abrirModalEditar(item: DetalleOperacion): void {
    this.modoEdicion = true;
    this.operacionEdicion = { ...item };
    this.mostrarModal = true;
  }

  cerrarModal(): void {
    this.mostrarModal = false;
  }

  guardar(): void {
    if (!this.operacionEdicion.descripcion || !this.operacionEdicion.idCabeceraOperacion) {
      alert('Complete la descripción y seleccione un cliente.');
      return;
    }

    if (this.modoEdicion && this.operacionEdicion.idDetalleOperacion) {
      this.ejecucionesService.actualizarEjecucion(this.operacionEdicion.idDetalleOperacion, this.op
        next: () => { this.cerrarModal(); this.cargarDatos(); },
        error: (err) => alert(err.message)
      });
    } else {
      this.ejecucionesService.crearEjecucion(this.operacionEdicion).subscribe({
        next: () => { this.cerrarModal(); this.cargarDatos(); },
        error: (err) => alert(err.message)
      });
    }
  }

  eliminar(id: number | undefined): void {
    if (!id) return;
    if (confirm(`¿Eliminar operación #${id}?`)) {
      this.ejecucionesService.eliminarEjecucion(id).subscribe({
        next: () => this.cargarDatos(),
        error: (err) => alert(err.message)
      });
    }
  }

  limpiarFormulario(): DetalleOperacion {
    return {
      descripcion: '',
      cantidad: 1,
      importe: 0,
      idCabeceraOperacion: this.cabeceras.length > 0 ? this.cabeceras[0].idCabeceraOperacion : 1
    };
  }
}
```

4.6 Vista con Barra Lateral, Tablas y Modales (`panel.html`)

Ubicación: `src/app/components/panel/panel.html`

```
<div class="d-flex min-vh-100 bg-light">

  <!-- 1. FRANJA LATERAL (SIDEBAR) -->
  <aside class="sidebar bg-dark text-white p-3 d-flex flex-column shadow" style="width: 260px; min-
    <div class="sidebar-header pb-3 mb-3 border-bottom border-secondary">
      <div class="d-flex align-items-center">
        <i class="bi bi-stack fs-3 text-primary me-2"></i>
        <div>
          <h5 class="fw-bold mb-0 text-white">Gestión App</h5>
          <small class="text-secondary">.NET 10 + Angular</small>
        </div>
      </div>
    </div>

    <!-- Menú de Selección -->
    <div class="nav nav-pills flex-column mb-auto gap-2">
      <div class="text-uppercase small fw-bold text-secondary px-2 mt-2 mb-1">Módulos</div>

      <!-- Botón Detalles -->
      <button (click)="cambiarSeccion('detalles')"
              class="nav-link text-start d-flex justify-content-between align-items-center"
              [class.active]="seccionActiva === 'detalles'"
              [class.text-white-50]="seccionActiva !== 'detalles'">
        <span><i class="bi bi-list-task me-2 fs-5"></i>Detalles de Operación</span>
        <span class="badge bg-primary rounded-pill">{{ ejecuciones.length }}</span>
      </button>

      <!-- Botón Cabeceras -->
      <button (click)="cambiarSeccion('cabeceras')"
              class="nav-link text-start d-flex justify-content-between align-items-center"
              [class.active]="seccionActiva === 'cabeceras'"
              [class.text-white-50]="seccionActiva !== 'cabeceras'">
        <span><i class="bi bi-building me-2 fs-5"></i>Cabeceras (Clientes)</span>
        <span class="badge bg-info text-dark rounded-pill">{{ cabeceras.length }}</span>
      </button>
    </div>

    <div class="sidebar-footer pt-3 border-top border-secondary text-secondary small">
      API: <strong>localhost:5264</strong>
    </div>
  </aside>

  <!-- 2. CONTENIDO PRINCIPAL -->
  <main class="flex-grow-1 p-4 overflow-auto">
    <div class="d-flex justify-content-between align-items-center mb-4">
      <h2 class="fw-bold text-dark mb-0">
        {{ seccionActiva === 'detalles' ? 'Listado de Detalles de Operación' : 'Listado de Cabeceras
      </h2>
      @if (seccionActiva === 'detalles') {
        <button (click)="abrirModalCrear()" class="btn btn-primary">
          <i class="bi bi-plus-circle me-1"></i> Nueva Operación
        </button>
      }
    </div>

    <!-- Filtro y Paginador -->
    <div class="card mb-3 shadow-sm border-0">
      <div class="card-body py-2">
        <input type="text" [(ngModel)]="filtroTexto" (ngModelChange)="paginaActual = 1" class="for
      </div>
    </div>

    <!-- TABLAS DINÁMICAS -->
    <div class="card shadow-sm border-0">
      <div class="card-body p-0">
        @if (cargando) {
```

```
          <div class="text-center py-5"><div class="spinner-border text-primary"></div></div>
        } @else {

          <!-- Tabla Detalles -->
          @if (seccionActiva === 'detalles') {
            <div class="table-responsive">
              <table class="table table-hover align-middle mb-0">
                <thead class="table-light">
                  <tr>
                    <th>#ID</th>
                    <th>Cliente</th>
                    <th>Descripción</th>
                    <th class="text-center">Cant.</th>
                    <th class="text-end">Importe</th>
                    <th class="text-end">IGV (18%)</th>
                    <th class="text-end">Total</th>
                    <th class="text-center">Acciones</th>
                  </tr>
                </thead>
                <tbody>
                  @for (item of detallesPaginados; track item.idDetalleOperacion) {
                    <tr>
                      <td class="fw-bold">#{{ item.idDetalleOperacion }}</td>
                      <td>{{ item.cabeceraOperacion?.cliente }}</td>
                      <td>{{ item.descripcion }}</td>
                      <td class="text-center">{{ item.cantidad }}</td>
                      <td class="text-end">{{ item.importe | currency:'PEN':'S/. ' }}</td>
                      <td class="text-end">{{ item.igv | currency:'PEN':'S/. ' }}</td>
                      <td class="text-end fw-bold text-success">{{ item.total | currency:'PEN':'S/.
                      <td class="text-center">
                        <button (click)="abrirVerMasDetalle(item)" class="btn btn-sm btn-outline-i
                        <button (click)="abrirModalEditar(item)" class="btn btn-sm btn-outline-pri
                        <button (click)="eliminar(item.idDetalleOperacion)" class="btn btn-sm btn-
                      </td>
                    </tr>
                  }
                </tbody>
              </table>
            </div>
          }

          <!-- Tabla Cabeceras -->
          @if (seccionActiva === 'cabeceras') {
            <div class="table-responsive">
              <table class="table table-hover align-middle mb-0">
                <thead class="table-light">
                  <tr>
                    <th>#ID</th>
                    <th>Cliente</th>
                    <th>Registro</th>
                    <th class="text-center">Operaciones</th>
                    <th class="text-center">Acciones</th>
                  </tr>
                </thead>
                <tbody>
                  @for (cab of cabecerasPaginadas; track cab.idCabeceraOperacion) {
                    <tr>
                      <td class="fw-bold">#{{ cab.idCabeceraOperacion }}</td>
                      <td>{{ cab.cliente }}</td>
                      <td>{{ cab.registro | date:'dd/MM/yyyy' }}</td>
                      <td class="text-center">{{ obtenerOperacionesPorCabecera(cab.idCabeceraOperac
                      <td class="text-center">
                        <button (click)="abrirVerMasCabecera(cab)" class="btn btn-sm btn-outline-p
                      </td>
                    </tr>
                  }
                </tbody>
              </table>
```

```
            </div>
          }

        }
      </div>

      <!-- Paginación -->
      <div class="card-footer bg-white d-flex justify-content-between align-items-center py-2">
        <small class="text-muted">Página {{ paginaActual }} de {{ totalPaginas }}</small>
        <div class="btn-group">
          <button (click)="cambiarPagina(paginaActual - 1)" [disabled]="paginaActual === 1" class
          <button (click)="cambiarPagina(paginaActual + 1)" [disabled]="paginaActual >= totalPagi
        </div>
      </div>
    </div>
  </main>
</div>

<!-- 3. MODAL VER MÁS / DETALLAR -->
@if (mostrarModalVerMas) {
  <div class="modal fade show d-block" style="background: rgba(0,0,0,0.5);">
    <div class="modal-dialog modal-dialog-centered modal-lg">
      <div class="modal-content">
        <div class="modal-header bg-dark text-white">
          <h5 class="modal-title">Detalle Completo</h5>
          <button (click)="cerrarModalVerMas()" class="btn-close btn-close-white"></button>
        </div>
        <div class="modal-body p-4">
          @if (detalleSeleccionado) {
            <p><strong>Cliente:</strong> {{ detalleSeleccionado.cabeceraOperacion?.cliente }}</p>
            <p><strong>Descripción:</strong> {{ detalleSeleccionado.descripcion }}</p>
            <p><strong>Cantidad:</strong> {{ detalleSeleccionado.cantidad }}</p>
            <p><strong>Importe:</strong> {{ detalleSeleccionado.importe | currency:'PEN':'S/. ' }}<
            <p><strong>IGV (18%):</strong> {{ detalleSeleccionado.igv | currency:'PEN':'S/. ' }}</p
            <h4><strong>Total:</strong> {{ detalleSeleccionado.total | currency:'PEN':'S/. ' }}</h4
          }
        </div>
        <div class="modal-footer"><button (click)="cerrarModalVerMas()" class="btn btn-secondary">(
      </div>
    </div>
  </div>
}

<!-- 4. MODAL CREAR / EDITAR -->
@if (mostrarModal) {
  <div class="modal fade show d-block" style="background: rgba(0,0,0,0.5);">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content">
        <div class="modal-header bg-primary text-white">
          <h5 class="modal-title">{{ modoEdicion ? 'Editar' : 'Nueva Operación' }}</h5>
          <button (click)="cerrarModal()" class="btn-close btn-close-white"></button>
        </div>
        <div class="modal-body p-3">
          <div class="mb-2">
            <label class="form-label">Cliente:</label>
            <select [(ngModel)]="operacionEdicion.idCabeceraOperacion" class="form-select">
              @for (c of cabeceras; track c.idCabeceraOperacion) {
                <option [value]="c.idCabeceraOperacion">{{ c.cliente }}</option>
              }
            </select>
          </div>
          <div class="mb-2">
            <label class="form-label">Descripción:</label>
            <input type="text" [(ngModel)]="operacionEdicion.descripcion" class="form-control">
          </div>
          <div class="row g-2 mb-2">
            <div class="col-6"><label class="form-label">Cantidad:</label><input type="number" [(ng
            <div class="col-6"><label class="form-label">Importe Base:</label><input type="number"
```

```
          </div>
        </div>
        <div class="modal-footer">
          <button (click)="cerrarModal()" class="btn btn-secondary">Cancelar</button>
          <button (click)="guardar()" class="btn btn-primary">Guardar</button>
        </div>
      </div>
    </div>
  </div>
}
```

4.7 Estilos del Sidebar (`panel.css`)

Ubicación: `src/app/components/panel/panel.css`

```
.sidebar {
  min-height: 100vh;
}

.sidebar .nav-link {
  color: #adb5bd;
  border-radius: 8px;
  padding: 10px 14px;
}

.sidebar .nav-link:hover {
  background-color: rgba(255, 255, 255, 0.08);
  color: #ffffff;
}

.sidebar .nav-link.active {
  background-color: #0d6efd;
  color: #ffffff;
}
```

5. Tabla Rápida de Reemplazo para el Examen

Si en tu examen te piden:	En Backend C#	En Model TS (`.model.ts`)	En Servicio (`.service.ts`)	En Componente (`panel.ts/.html`)
Cabecera (Padre)	`CabeceraOperacion` / `Cliente`	`cabecera.model.ts`	`getCabeceras()`	Menú Lateral "Cabeceras"
Detalle (Hijo)	`DetalleOperacion` / `Venta`	`detalle.model.ts`	`getEjecuciones()` , `crear()` , etc.	Menú Lateral "Detalles"
Acción "Ver Más"	Endpoint GET `{id}` o relación	`abrirVerMasDetalle(item)`	Muestra modal con datos desglosados	Botón `<button> Ver más</button>`

6. Top 5 Errores Típicos de Examen y Soluciones en 10 Segundos

1. **CORS bloqueado en consola (`Access to fetch... blocked by CORS policy`)**:
2. En `Program.cs` de .NET falta `app.UseCors("AngularPolicy");` o la política no tiene `http://localhost:4200`.
3. `NullInjectorError: No provider for _HttpClient`:
4. Falta `provideHttpClient()` dentro de `providers: [...]` en `app.config.ts`.
5. `Property 'id...' is missing in type but required`:
6. Olvidaste poner el signo `?` en el ID de la interfaz TypeScript (`idDetalleOperacion?: number;`).
7. `Can't bind to 'ngModel' since it isn't a known property of 'input'`:
8. Falta importar `FormsModule` en el array `imports: [CommonModule, FormsModule]` de `panel.ts`.
9. **La tabla no se refresca al guardar o eliminar**:
10. Asegúrate de llamar a `this.cargarDatos()` dentro del bloque `next: () => { this.cargarDatos(); }` del `.subscribe()`.